



Backend Developer Assignment: Stock Trading Web Portal (Node.js)

Objective:

Develop a **Stock Trading Web Portal** using **Node.js**. This portal should allow the entry and management of trades and implement **Lot Management** using both **FIFO (First-In-First-Out)** and **LIFO (Last-In-First-Out)** methods.

Requirements:

- Develop the backend using **Node.js (Express.js)**.
 - Use **MongoDB / PostgreSQL** as the database.
 - Implement REST APIs for CRUD operations.
 - Ensure that the data is properly structured and follows a well-defined schema.
 - Properly handle FIFO and LIFO calculations when handling reverse (negative quantity) trades.
-

Functional Modules

1. Trade Module:

- This module will handle all trade entries.
- Each trade will have the following attributes:
 - trade_id (Unique identifier)
 - stock_name (e.g., Apple, Tesla, etc.)
 - quantity (Can be positive for buy or negative for sell)
 - broker_name (Broker through which the trade was made)
 - price (Price per unit stock)
 - amount (Total value of trade = price * quantity)
 - timestamp (Date and time of trade execution)
- **Example Trade Entries:**



- **Trade 1:** Buy 100 shares of Apple at \$150 each through Broker A.
- **Trade 2:** Buy 200 shares of Apple at \$160 each through Broker B.
- **Trade 3:** Sell 150 shares of Apple at \$170 each through Broker A.

2. Lot Module (FIFO Method):

- Each stock (new symbol) creates a **new lot**.
- Additional purchases of the same stock create additional lots.
- Selling stocks **realizes** existing lots based on **FIFO**.
- Attributes:
 - lot_id (Unique identifier)
 - trade_id (Reference to trade that created this lot)
 - lot_quantity (Total quantity in the lot)
 - realized_quantity (Quantity that has been sold from this lot)
 - realized_trade_id (Trade ID of the trade that realized this lot)
 - lot_status (OPEN / PARTIALLY REALIZED / FULLY REALIZED)

- **Example FIFO Execution:**

1. **Buy 100 shares** of Apple at \$150 → **Lot 1 created**.
2. **Buy 200 shares** of Apple at \$160 → **Lot 2 created**.
3. **Sell 150 shares** of Apple at \$170 →
 - **Lot 1 (100 shares) fully realized**.
 - **Lot 2 (50 shares) partially realized**.

3. Lot Module (LIFO Method):

- Each stock (new symbol) creates a **new lot**.
- Additional purchases of the same stock create additional lots.
- Selling stocks **realizes** existing lots based on **LIFO**.
- Attributes:



- lot_id (Unique identifier)
- trade_id (Reference to trade that created this lot)
- lot_quantity (Total quantity in the lot)
- realized_quantity (Quantity that has been sold from this lot)
- realized_trade_id (Trade ID of the trade that realized this lot)
- lot_status (OPEN / PARTIALLY REALIZED / FULLY REALIZED)

- **Example LIFO Execution:**

1. **Buy 100 shares** of Apple at \$150 → **Lot 1 created.**
2. **Buy 200 shares** of Apple at \$160 → **Lot 2 created.**
3. **Sell 150 shares** of Apple at \$170 →
 - **Lot 2 (200 shares) realizes 150 shares first.**
 - **Lot 2 now has 50 shares remaining.**

Deliverables:

- **API Endpoints:** RESTful APIs for:
 - Creating and fetching trades.
 - Fetching lot details (FIFO & LIFO calculations).
 - Handling reverse trades properly (matching them against existing lots).
- **Database Schema:**
 - Design and implement schema for trades and lots.
- **Code Documentation:**
 - Properly document code with comments.
 - Provide README with setup instructions.
- **Test Cases:**
 - Unit test cases for FIFO & LIFO logic.



- **Loader Capability:**

- Implement a bulk loader to add trades in the Trade module.

- **Deployment:**

- The application must be deployed.
- Share the deployment URL along with login credentials (ID and password).

Evaluation Criteria:

- **Code Quality:** Clean, modular, and well-commented code.
- **Database Design:** Proper schema and relationships.
- **FIFO & LIFO Implementation:** Accurate lot handling.
- **API Performance:** Efficient query handling.
- **Error Handling:** Proper handling of edge cases.

Good luck, and happy coding!